Asignación de Pickers

La asignación distribuye las unidades pendientes de las waves entre los pickers disponibles. Considera la capacidad de carga, las zonas de despacho y la distancia estimada de los viajes. Utiliza búsqueda exacta para rondas pequeñas y una estrategia voraz para rondas grandes, sin garantizar una solución global óptima.

1. Asignación de pickers (AsignadorPickers.Asignador)

Idea

Seleccionar qué unidades puede preparar cada picker y a qué zona de despacho debe entregarlas, buscando asignar la mayor cantidad factible y comparar los costos estimados de los viajes. El asignador administra los pendientes y crea los trabajos resultantes.

Estructuras del código

- Diccionarios por wave: guardan cantidades solicitadas y pendientes, trabajos activos y estado de preparación.
- Diccionarios por zona: registran capacidad, peso reservado, peso entregado y celdas de despacho.
- Listas de unidades, asignaciones y cargas: representan las alternativas de una ronda para los pickers disponibles.
- Vectores de bits y estados de búsqueda: permiten comparar alternativas en el backtracking.
- Caché de planes: reutiliza evaluaciones de viajes para un picker y un conjunto de productos durante la ronda.

Paso a paso

1. En la inicialización, recibe las waves, los pickers por asignar posteriormente, las zonas, los productos, los racks y la navegación. Normaliza los productos de cada wave y prepara el estado de las zonas.
2. La función asignar(libres) ordena los pickers libres por identificador y recorre las waves con unidades pendientes. Para cada wave llama a _resolver_ronda e impide asignar más de un nuevo viaje al mismo picker.
3. _resolver_ronda expande las cantidades pendientes en unidades individuales y estima las combinaciones posibles: (P + 1)^Q, donde P es la cantidad de pickers candidatos y Q la cantidad de unidades. El término adicional representa dejar una unidad sin asignar.
4. Si el número estimado no supera max_combinaciones_exactas (5000 por defecto), _buscar_asignaciones utiliza backtracking: para cada unidad prueba pickers con capacidad suficiente o la opción de omitirla. Evita algunas permutaciones equivalentes y poda alternativas inferiores.
5. Para cada asignación completa factible, _opciones_de_viaje consulta al planificador; _elegir_zonas determina las zonas de entrega respetando su capacidad. Con hasta ocho pickers activos se exploran combinaciones de zonas; con más, se usa una elección voraz.
6. Si se supera el umbral de combinaciones, _resolver_ronda_voraz procesa las unidades en orden. Prueba agregarlas a cada picker factible y conserva la alternativa con mejor evaluación de viajes y zonas en ese paso.

7. _crear_trabajos agrupa los productos asignados, descuenta las unidades pendientes, reserva peso en despacho y genera trabajos con picker, wave, productos, zona y distancia estimada.
8. Cuando se confirma una entrega, trabajo_entregado actualiza el peso reservado y entregado y el estado de la wave. El asignador también identifica unidades que no pueden prepararse.

Complejidad

La búsqueda exacta explora un árbol de hasta (P + 1)^Q alternativas (antes de podas). Por cada hoja factible, puede evaluar hasta P pickers y Z zonas y ejecutar la selección de zonas. Su cota completa se desarrolla en la sección 4. El límite configurable de 5000 combinaciones restringe cuándo se utiliza esta rama, pero no elimina el costo de las evaluaciones de viajes.

2. Planificación de viajes (PlanificadorViajes.PlanificadorViajesV)

Idea

Evaluar la factibilidad y distancia de un viaje propuesto para un picker, desde su posición, pasando por los accesos a los productos y terminando en una celda de despacho. El planificador estima los viajes, pero no decide cómo repartir las unidades entre los pickers.

Estructuras del código

- Diccionarios de productos por acceso y racks por acceso: reúnen productos que se recogen desde una misma ubicación accesible.
- Listas de nodos de visita y de productos: conservan los puntos necesarios y el orden resultante.
- Caché de grafos: reutiliza el grafo de planificación según posición inicial y celdas de despacho.
- Caché de caminos: evita repetir búsquedas entre pares de nodos de un mismo grafo.

Paso a paso

1. La clase recibe la navegación, los productos y racks, el algoritmo de búsqueda de caminos y la estrategia de recorrido. Valida que ambas estrategias sean conocidas.
2. evaluar_viaje recibe un picker, los productos seleccionados, su orden de referencia, una zona y sus celdas de despacho posibles.
3. Agrupa los productos por acceso. Si falta un acceso requerido o no hay productos para visitar, descarta el viaje.
4. Construye o recupera el grafo de planificación; intenta utilizar un grafo reducido y, si no es posible, recurre al grafo completo.
5. Para cada celda de despacho candidata aplica la estrategia de orden de visita configurada (orden, cercanía o zonas), que solicita caminos con Dijkstra o A* mediante _buscar_con_cache.
6. Reconstruye el orden de productos y descarta planes que no cubran todos los solicitados. Compara los candidatos por distancia total y desempata por celda de destino.
7. Devuelve el mejor plan válido, con distancia, zona, destino y orden de productos; si no encuentra uno factible, devuelve None.

Complejidad

Su costo puede acotarse en función de L (líneas de la wave), S (accesos distintos del viaje), D (celdas de despacho), G (preparación del grafo para esa clave) y B (costo de una búsqueda de camino, incluida la reconstrucción): $F = O(L + G + D \cdot [L + S^2 \cdot B + S^2 \cdot V + S \log S])$. Para orden y zonas

se realizan O(S) búsquedas por destino, mientras que cercanía puede realizar $O(S^2)$. La expresión conservadora usa la opción más costosa. Si el grafo o los caminos están en caché, sus costos de construcción/búsqueda se evitan en los aciertos.

3. Cómo se utilizan en la asignación

AsignadorPickers decide los productos y pickers candidatos y solicita a PlanificadorViajes una estimación para cada viaje que necesita comparar. Con esos planes verifica las zonas disponibles, elige la distribución y genera los trabajos. Así se separa la decisión de asignación del cálculo de recorridos, sin incluir módulos cuya responsabilidad principal sea ejecutar la simulación.

4. Complejidad general

Se analizan la inicialización del asignador, una ronda de asignación y las llamadas al planificador. Todas las cotas son de peor caso, salvo operaciones esperadas O(1) en diccionarios y conjuntos. Los límites configurables son 5000 combinaciones para activar la búsqueda exacta y 8 pickers activos para la selección exacta de zonas.

Variables utilizadas

- L: cantidad de líneas de productos de todas las waves
- L_v: líneas de la wave evaluada
- Q: unidades pendientes en la ronda
- P: pickers candidatos
- A: pickers activos ($A \leq P$)
- Z: zonas de despacho.
- D: cantidad máxima de celdas de despacho por zona
- S: accesos de productos distintos que debe visitar un viaje ($S \leq L_v$)
- V y E: vértices y aristas del grafo usado en planificación.
- B: costo de calcular y reconstruir un camino sobre ese grafo
- G: costo de preparar el grafo para una combinación de posición inicial y celdas de despacho (si no está en caché).
- F: costo máximo de una evaluación PlanificadorViajes.evaluar_viaje para una zona
- H(A,Z): costo de elegir zonas para A pickers activos
- C_p, Cg y C_r: espacio acumulado en cachés de planes, grafos y caminos.

Costo de evaluar un viaje: PlanificadorViajes

La función agrupa los productos por accesos recorriendo el orden de la wave: $O(L_v)$. La planificación puede preparar o recuperar un grafo (G). Para cada una de las D celdas de despacho, calcula un recorrido sobre S accesos, reconstruye el orden de productos y compara el resultado.

La estrategia “orden” hace O(S) búsquedas de caminos por destino; “zonas” también, más O(S log S) por ordenación; “cercanía” puede hacer $O(S^2)$ búsquedas por destino. Cada búsqueda nueva tiene costo B. El armado de la ruta concatena caminos de hasta V vértices, por lo que su costo también debe considerarse.

Cota conservadora para una zona: $F = O(L_v + G + D \cdot (L_v + S^2(B + V) + S \log S))$.

Dijkstra y A* con heap pueden acotarse en el peor caso por $B = O((V + E) \log(V + 1) + V)$, incluyendo reconstrucción. Las búsquedas repetidas que encuentran su camino en caché evitan ese

costo de búsqueda; G también desaparece cuando el grafo ya está disponible. Esta fórmula es una cota superior, no el tiempo típico de ejecución.

Costo de elegir zonas: _elegir_zonas

Con A ≤ 8 se usa _buscar_zonas. Cada picker puede elegir hasta Z zonas: hay hasta Z^A combinaciones, sujetas a restricciones y podas. En la implementación, comprobar cada nodo y construir la clave final puede requerir $O(A^2)$ en el peor caso por el recálculo de distancias y las rebanadas de listas.

$H(A,Z) = O(A^2 \cdot Z^A)$ si A ≤ 8. Para A > 8 se usa la elección voraz: ordena los pickers y sus opciones de zona; $H(A,Z) = O(A \log A + AZ \log Z + A^2)$.

Aunque la selección exacta de zonas tiene crecimiento exponencial respecto de A, su implementación restringe esa rama a un máximo de ocho pickers activos.

Búsqueda exacta: _buscar_asignaciones

Cada una de las Q unidades puede ir a uno de los P pickers o quedar sin asignar: hasta (P + 1)^Q hojas y un número de nodos del mismo orden (para P ≥ 1). Las podas y la simetría de unidades repetidas pueden reducir esa cantidad, pero no cambian la cota superior.

En una hoja factible se recorren los pickers activos, se consultan hasta P · Z planes de viaje y se eligen zonas. La caché reutiliza evaluaciones anteriores del mismo picker y conjunto de productos; para obtener una cota independiente de los aciertos se supone que cada consulta requiere evaluar un viaje.

$T_{exacta}(Q,P) = O((P + 1)^Q \cdot [Q + PZ(F + 1) + H(P,Z)])$.

La expresión usa H(P,Z) como cota de H(A,Z), entendida por tramos según el número de activos. En esta implementación, la búsqueda exacta solo se ejecuta cuando (P + 1)^Q ≤ C, con C = 5000 por defecto. Por lo tanto, para C fijo el número de alternativas está acotado, aunque el costo F de los viajes siga dependiendo del grafo.

Asignación voraz: _resolver_ronda_voraz

Para cada unidad se prueban hasta P pickers. En cada alternativa se vuelven a revisar hasta P pickers activos, sus opciones para Z zonas y la selección de zonas. Además, construir las listas de activos y los estados provisionales cuesta O(P).

$T_{voraz}(Q,P) = O(QP \cdot [PZ(F + 1) + P + H(P,Z)] + Q + P)$.

Esta cota sobreestima los casos con cachés reutilizadas. En particular, no se puede simplificar el algoritmo completo a O(QP), porque una alternativa exige verificar viajes factibles y zonas de despacho.

Inicialización, creación de trabajos y coordinación

La inicialización consolida líneas de waves y registra zonas y celdas de despacho: $O(L + Z + D_t)$, más el costo de obtener accesos faltantes. D_t representa el total de celdas de despacho registradas. Crear los trabajos de una ronda requiere O(Q + P) en el peor caso. En asignar(libres) se ordenan los pickers en O(P log P) y se recorren hasta W waves; la suma de las rondas procesadas, no una única Q global, determina el costo total de una llamada.

Resumen de complejidad temporal

Etapa	Tiempo	Espacio
Planificador (una zona)	$F = O(L_v + G + D[L_v + S^2(B + V) + S \log S])$	$O(L_v + S + \text{ruta})$, sin cachés
Selección de zonas	$H = O(A^2 Z^A)$ para $A \leq 8$; voraz para $A > 8$	$O(A + Z)$, sin planes
Búsqueda exacta	$O((P+1)^Q [Q + PZ(F+1) + H])$	$O(Q + P + Z)$, sin cachés
Asignación voraz	$O(QP [PZ(F+1) + P + H] + Q + P)$	$O(Q + P + Z)$, sin cachés
Inicialización	$O(L + Z + D_t)$, salvo accesos	$O(L + Z + D_t)$

Espacio total y fórmula por ronda

Las estructuras activas de la búsqueda usan $O(Q + P + Z)$ de espacio auxiliar. A ello se agregan los estados persistentes de las waves, $O(L + Z + D_t)$, y las cachés: C_p (planes), Cg (grafos) y C_r (caminos, que pueden almacenar secuencias de hasta V vértices). Estas cachés crecen con las combinaciones distintas que se evalúan y no tienen en el código una cota fija de tamaño.

Espacio total: $O(L + Z + D_t + Q + P + C_p + Cg + C_r)$.

Si $E(Q,P)$ denota la fórmula exacta y $R(Q,P)$ la voraz, el costo de resolver una ronda es $O(Q + P + E(Q,P))$ cuando $(P+1)^Q \leq C$, y $O(Q + P + R(Q,P))$ en caso contrario. El costo de asignar(libres) suma las rondas efectivamente ejecutadas para las waves consideradas, más $O(P \log P + W \cdot P)$ de coordinación.

5. Conclusiones

La asignación combina restricciones de peso y despacho con la evaluación de distancias estimadas. Su costo no depende solamente de Q y P: las zonas y las búsquedas sobre el grafo pueden dominar cada alternativa. La búsqueda exacta tiene una cota exponencial, pero se utiliza solo bajo el umbral configurado; la voraz reduce las alternativas de distribución sin evitar el costo de evaluar viajes. Las podas y las cachés mejoran el rendimiento real sin modificar las cotas conservadoras anteriores.